TITLE: A Deterministic Parallel Simulator of a Coherent Multi-SM GPU (Devika Manohar, Barbara Zhuo)

URL: https://github.com/dmanohar2110/parallel-gpu-simulator

SUMMARY:

We plan to build a simulator of a multi-SM GPU's memory system (SMs, crossbar, L2 slices, DRAM) and parallelize the simulator itself. Threads will simulate independently for windows as long as the interconnect latency, then synchronize, so the results exactly match a sequential version. We will run it with C++ threads (OpenMP) on the 8-core GHC machines and the 128-core PSC Bridges-2 nodes.

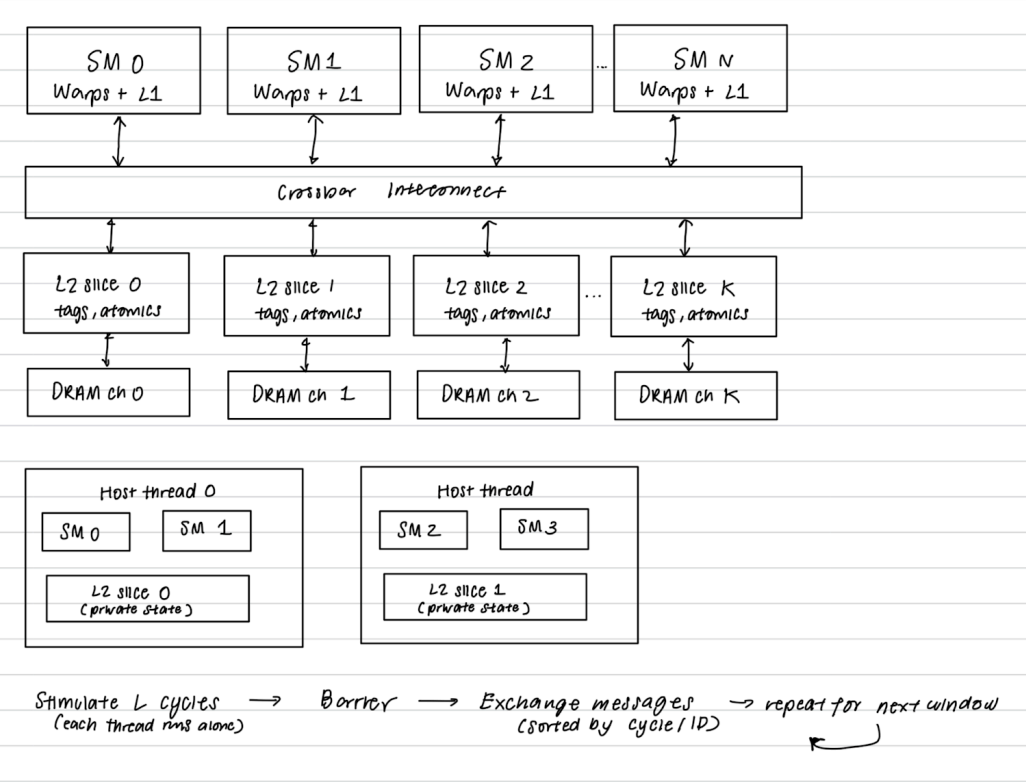
BACKGROUND:

What the application is and why it is slow:

- Architects use cycle-level simulators to evaluate GPU designs before building hardware. For example, a C++ golden model of a GPU that is used in verification testbenches to compare results against the RTL.
- They're typically slow because every simulated cycle, the simulator has to update every SM, warp, cache, and queue. Simulating even a short kernel can take millions of cycles and lots of components.
- Most simulators are sequential (because you care most about the correctness of the result they produce), so we want to create one that is both correct and fast.

How it works:

Our simulated GPU uses GPU-style coherence rather than a CPU protocol like MESI. Each SM's L1 cache is write-through and is invalidated at synchronization points (fences and kernel boundaries). So, the L2 cache is where the data is always up to date.



- Pseudocode for the main loop of the design:

```
C/C++
for cycle = 0 . . . final_cycle:
      for each SM:
            1. issue warp memory operations
            2. send memory requests
      for each L2 cache section:
            1. process requests
            2. send replies
```

What could benefit from parallelism and why:

- Within a cycle, each SM's update and each L2 slice's update only touches its own state, so those updates are independent. They interact through messages, and messages take the interconnect latency to arrive.
- So different threads can simulate different components, and synchronize when messages are exchanged.

THE CHALLENGE:

What aspects of the problem might make it difficult to parallelize?

- There are lots of SMs and L2 slices, so you can give each thread a few. However, the main challenge is that they keep sending messages to each other, so threads can't just run on their own. They're only independent for short stretches, then have to coordinate.
- The parallel simulator has to give exactly the same results as the sequential one, every run, for any number of threads in order to be a trustworthy and deterministic golden model. A version that is fast but gives slightly different numbers each time does not meet the requirement. The easiest way to go faster is to sync less often and let threads run ahead. But then some messages arrive late, after the receiving thread has already simulated past that cycle, so the results come out wrong.

Describe the workload: what are the dependencies, what are its memory access characteristics? (Is there locality? Is there a high communication to computation ratio? Is there divergent execution?)

- There are several workload dependencies which include:
    - Messages are sent across components every few simulated cycles. One memory load is a request from an SM to an L2 slice and a reply back, and the SM must wait for the L2 to process the request and reply.
    - The ordering of messages dispatched in the same cycle matters. In the sequential simulator, the order is always the same, but in the parallel version, the requests come from different threads, so whichever thread delivers first would win, and the order could change between runs.
- The memory access characteristics are:
    - Each thread has its own private memory because each thread simulates a few SMs and L2 slices, and it's the only one that ever touches their data, so it stays in that core's cache and access is fast. However, the locality of the addresses being accessed depends on the trace.
    - Messages between threads involve shared memory, and are slower (written by one core and read by another). The data has to move between the cores' caches, which is much slower.
- There is a high communication-to-computation ratio because there is typically very little work between interactions.
    - Simulating one SM for one cycle is fairly small because it involves checking a few queues and maybe issuing a request. However, a barrier across 8 threads costs much more time (especially across 128

threads). So, syncing every cycle would be expensive, and we would have to use some kind of lookahead logic to fix this.

Describe constraints: What are the properties of the system that make mapping the workload to it challenging?

- Thread timing on real hardware is non-deterministic, so which thread delivers a message first can vary from run to run. So, some sort of ordering guarantee needs to be implemented.
- Fixed core counts versus the number of simulated components. GHC has 8 cores and PSC has up to 128. If the simulated GPU has 32 SMs and 16 L2 slices, then at 128 threads some threads get almost nothing to do.
- Barrier costs also grow with the size of the machine we use. A barrier across 128 threads on multiple sockets costs a lot more than across 8 threads, so the same window size can work on GHC but not on PSC.

RESOURCES:

Machines we will be using:

- GHC machines (8 cores): development, debugging, and most experiments.
- PSC Bridges-2 RM nodes (128 cores, multi-socket): scaling experiments and NUMA effects.

Starter code:

- We will mainly be starting from scratch in C++ with OpenMP, but we would like to check whether Professor Railing's simulation framework (mentioned in the project-proposal.pdf document) can serve as a starting point.

References:

- Parallel Discrete Event Simulation: https://dl.acm.org/doi/epdf/10.1145/84537.84545
- Fast and Accurate Microarchitectural Simulation of Thousand-Core Systems: https://people.csail.mit.edu/sanchez/papers/2013.zsim.isca.pdf
- MGPUSim: Enabling Multi-GPU Performance Modeling and Optimization: https://dl.acm.org/doi/epdf/10.1145/3307650.3322230

GOALS AND DELIVERABLES:

Plan to achieve:

- A valid sequential reference
    - Make a correct sequential model of the multi-SM GPU memory system that includes the basics: SMs that replay warp memory traces (with a coalescer, L1 cache, and MSHRs), a crossbar interconnect, L2 cache slices, and DRAM channels.
    - This is achievable because we have an idea of how to map the hardware components to standard software data structures (e.g std::queue for FIFOs, structs for state machines). Because it is strictly single-threaded, we can develop and debug this baseline in C++ without having to manage race conditions/synchronization.
- Correctness in the parallelized version
    - Ensure deterministic execution: every parallel run must exactly match the sequential version, including the cycle each memory request completes, the order messages are processed, total cycle counts, and cache hit/miss statistics, for any number of threads.
    - Ensure components cannot physically affect one another faster than the interconnect delay. This also helps guarantee the parallel execution is identical to the sequential baseline.
- Performance Objective: N/2x speedup when code is run on a N-core machine (4x speedup on the 8-core GHC machines)

    - We anticipate difficulties with achieving a close to linear speedup, accounting for high barrier synchronization delays. For example, we see the case where if we assign 4 SMs to Thread A and 4 to Thread B, but a particular input only utilizes Thread A's SMs during a cycle window, Thread B will stall at the OpenMP barrier waiting for Thread A to finish. N/2x speedup provides a realistic margin for these load imbalances.
- Testing suite
    - Develop a test program and benchmarking suite that measures simulator throughput and breaks down execution time.

Hope to achieve:

- Dynamic Load Balancing
    - Instead of statically assigning a fixed number of SMs to each host thread, we can extend the project by trying to implement a dynamic assignment strategy that redistributes active SMs to host CPU threads at the window boundaries. This can improve our speedup and efficient resource usage.
- NUMA-Aware Memory
    - We can attempt to do NUMA-aware data allocation. This ensures the memory backing a specific SM resides on the same physical CPU socket computing it.
    - OpenMP has built-in environment controls (OMP_PLACES and OMP_PROC_BIND) that can pin threads to specific physical cores. This can minimize cross-socket memory traffic when updating shared queues.
- Relaxed synchronization
    - Let threads run for longer windows than the interconnect latency allows, so they hit barriers less often and the simulator runs faster.
    - The issue is that some messages now arrive late, after the receiving thread has already simulated past their arrival cycle, so the results are no longer exactly correct. We'd measure how much faster it gets and how far the results drift from the sequential golden model (for example, how different the total cycle counts are), to see whether the speedup is worth the loss in accuracy.

Fallback scope:

- Limit parallelization scaling target to 8-Core GHC Machines
    - If barrier costs on the 128-core Bridges-2 nodes are too big for our window size, we will scale back our performance objective to only demonstrate speedup on the single-socket GHC machines.
    - As we noticed in asst 4, multi-socket synchronization overhead is more difficult to control. Single-socket architectures have faster cache coherence protocols, so our parallelized model is more likely to succeed on the GHC machines if it fails to scale to 128 cores.

PLATFORM CHOICE:

We will be using C++ and OpenMP as the parallel framework. C++ being an object oriented language allows us to use software abstractions easily to map physical hardware components with its structs/class structures. Since simulation of a low level machine requires millions of state updates per second, it is more ideal to use a low-level language like C++.

Our simulated GPU relies on passing messages between SMs and L2 slices over a crossbar, so OpenMP's shared-memory model lets each thread write its outgoing messages into a dedicated queue for each sender-receiver pair, which the receiving thread reads at the window boundary, without the network overhead of distributed-memory frameworks like MPI. Our parallelization strategy also requires all active threads to synchronize exactly at the window boundaries. OpenMP provides low-overhead implementations (#pragma omp barrier) that help minimize synchronization costs from the barriers inside of the main loop.

SCHEDULE:

Dates	**Task Items**
Monday, November 9th	● Submit project proposal, and set up a repository. ● Email Professor Railing about his simulation framework.
November 10th - 12th	● Freeze the hardware model and configuration parameters (SM count, slices, latencies). ● Define and finalize the trace format.
November 13th - 15th	● Create a correct baseline sequential SM model: warp replay, coalescer, L1, MSHRs. ● Create a sequential crossbar, L2 slices, atomics, DRAM. ● Do an end to end sequential run on a small trace that we write and verify that it works as expected (coherent).
November 16th - 18th	● Build a checker that confirms parallel runs produce exactly the same results as the sequential version, and add timers to measure where the simulator spends its time. ● Add traces with different memory access patterns (random gather, reuse heavy, etc.) ● Finish validating and profiling the sequential golden model.
November 18th - 20th	● Implement a naive lockstep OpenMP version of the simulator. ● Give each sender-receiver pair its own message queue, and have receivers process incoming messages in a deterministic order. ● Verify using the checker that the lockstep implementation matches the golden sequential model at 1-8 threads.
November 20th - 22nd	● Implement conservative lookahead windows (most challenging part). ● Verify that the parallel simulator is still correct with lookahead. ● Obtain the first GHC speedup curves, and draft milestone report with these.
November 23rd	● Submit milestone report.
November 24th - 25th	● Write timer code and get the time breakdown per thread: simulating vs. barrier vs. message exchange to understand where the time goes, and what the main bottlenecks are. ● Do the first set of PSC machine runs (1-128 threads on the same workload).
November 26th - 29th (during Thanksgiving break)	● Fix any bugs, and potentially optimize message queue layout to avoid false sharing. ● Create a sweep script for threads x L x workload.
November 30th - December 2nd	● Implement load-balancing strategies for assigning SMs and slices to threads (NUMA-aware placement). ● Run full GHC and PSC sweeps with the script (check weak-scaling, and cache miss counts). ● Decide whether to pursue the stretch goals and which ones to pursue.
December 3rd - 6th	● Stretch goal of relaxed sync with accuracy-vs-speed measurements. ● Start producing final graphs, and begin report outline.

December 7th - 8th	● Freeze code, and rerun any final experiments. ● Finish the report results and analysis sections.
December 9th - 10th	● Wrap up the report (approach and lessons learned). ● Do the poster session pages.
December 11th	● Poster session presentation.